```
const boolean degreesF = true;

const int hot = 100;
const int cold = 40;
const int hotColor = 0;
const int coldColor = 170;
const int brightness = 255;
```

This block of code declares the variables that can be used to configure various settings in our sketch. The first of these turns on Fahrenheit temperatures using `true` and by using `false` will revert to Celsius temperatures. The variable `hot` and `cold` allows us to specify what we think is hot and cold for our location—hot in London is perhaps not the same as hot in Phoenix. With these values set, any temperature below the cold limit will remain blue while any temperature hotter than the hot value will stay red. Since it is entirely possible that hot being red and cold being blue might be too obvious for you, you can change the values for `hotColor` and `coldColor` to establish the two extreme colors in HSB values. Finally, the BlinkM can be insanely bright, so we have an option to specify a different brightness level that will dim the LED without changing its color.

```
Wire.begin();
stopScript(blinkM);
setFadeSpeed(blinkM, 1);
```

Because we are using several new functions in our sketch, the code in our `setup()` function is a little sparse. The first line starts the communication protocol that we will use for the BlinkM. Again, this will be covered in greater detail later in this book. The `stopScript()` and `setFadeSpeed()` functions are two of our custom functions used for setting options in the BlinkM. We'll talk about these in a moment, so let's move on to the code in our `loop()` function.

```
int hue = map(temperature(), hot, cold, hotColor, coldColor);
```

We begin here with mapping whatever value is returned from the `temperature()` function, as defined by the `hot` and `cold` endpoints, to whatever `hue` value will correspond to that temperature, as defined by the `hotColor` and `coldColor` variables. Because it is possible for these values to end up outside of this range, we will also need the `constrain()` function:

```
hue = constrain(hue, hotColor, coldColor);
```

This line will keep the LED from suddenly turning pink if the values were to get a little wonky (technically speaking). Once we have our `hue` value established, we need to send this out to our BlinkM as we did in the following:

```
fadeToHSB(blinkM, hue, 255, brightness);
delay(10000);
```

The function `fadeToHSB()` will send a command to the BlinkM, telling it to fade to whatever color is specified in the hue that we just worked out; a saturation of 255, although you could play with this number; and the brightness that we established at the beginning of our sketch. We then delay for 10 seconds just to keep the LED from flickering because of small fluctuations in temperature. Now let's look at our functions.